

IMPLEMENTASI ALGORITMA GREEDY PADA PENGEMBANGAN BOT CERDAS DALAM PERMAINAN TANK ROBOCODE

Tugas Besar

Diajukan sebagai syarat menyelesaikan mata kuliah Strategi Algoritma (IF2211) Kelas
RA
di Program Studi Teknik Informatika, Fakultas Teknologi Industri, Institut Teknologi
Sumatera



Oleh: Kelompok Neural Alpha

Bayu Setiawan 124140177

Rakha Daffa Tama Truski 124140196

Dosen Pengampu: Imam Eko Wicaksono, S.Si., M.Si.

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI**

INSTITUT TEKNOLOGI SUMATERA 2025

DAFTAR ISI

DAFTAR ISI	2
BAB I DESKRIPSI TUGAS	3
1.1 Latar Belakang Masalah	3
1.2 Tujuan	3
1.3 Spesifikasi Masalah dan Komponen Lingkungan	4
1.4 Aturan dan Batasan Tugas Besar	5
BAB II LANDASAN TEORI	6
2.1 Definisi dan Karakteristik Algoritma Greedy	6
2.2 Elemen-Elemen Pembentuk Algoritma Greedy	6
2.3 Mekanisme Kerja Program dan Komunikasi Hub Tank Royale	7
BAB III APLIKASI STRATEGI GREEDY	8
3.1 Fungsi Obyektif	8
3.2 Eksplorasi Alternatif Solusi Greedy	8
3.3 Analisis Efisiensi dan Efektivitas Solusi Greedy	10
3.4 Strategi Greedy yang Dipilih	11
BAB IV IMPLEMENTASI DAN PENGUJIAN	12
4.1 Implementasi Algoritma Greedy	12
4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur	13
4.3 Skenario dan Hasil Pengujian	14
4.4 Analisis Hasil Pengujian	14
BAB V KESIMPULAN DAN SARAN	16
5.1 Kesimpulan	16
5.2 Saran	17
LAMPIRAN	19
DAFTAR PUSTAKA	20

BAB I DESKRIPSI TUGAS

1.1 Latar Belakang Masalah

Robocode Tank Royale merupakan sebuah platform simulasi pertempuran berbasis pemrograman objek (*Object-Oriented Programming Simulation*) yang dirancang untuk menguji keandalan logika berpikir developer melalui pembuatan kode otomatis untuk tank virtual. Dalam lingkungan simulasi ini, tangki-tangki virtual dilepaskan di dalam sebuah arena pertempuran dua dimensi tertutup dan saling berkompetisi hingga hanya tersisa satu entitas tank tunggal yang mampu bertahan hidup (*Battle Royale system*). Sesuai dengan namanya, Robocode (akronim dari *Robot code*), pengguna bertindak sepenuhnya sebagai perancang kecerdasan intelektual agen (programmer) tanpa adanya kendali manual secara langsung (*real-time user interaction*) ketika simulasi pertempuran dimulai.

Tantangan utama di dalam pengembangan bot pada Robocode Tank Royale adalah bagaimana mengolah masukan sensoris lingkungan yang sangat dinamis, terbatas, dan terus berubah di setiap satuan waktu terkecil (*turn*) menjadi serangkaian keputusan aksi yang optimal. Keputusan aksi tersebut meliputi pergerakan lambung robot (*body movement*), rotasi radar memutar arena (*scanning*), serta kalkulasi arah dan kekuatan tembakan laras meriam (*gun firing mechanics*). Untuk menyelesaikan masalah optimasi multi-variabel ini secara efisien tanpa membebani daya komputasi di setiap gilirannya, pendekatan berbasis Strategi Algoritma Greedy menjadi pilihan yang paling relevan untuk dieksplorasi.

1.2 Tujuan

1. Merancang dan mengimplementasikan bot otomatis pada platform Robocode Tank Royale menggunakan bahasa pemrograman C# (.NET) yang menerapkan prinsip algoritma *greedy* sebagai dasar pengambilan keputusan agen.
2. Memetakan elemen-elemen permainan Robocode Tank Royale — meliputi himpunan kandidat, fungsi seleksi, fungsi kelayakan, dan fungsi objektif — ke dalam kerangka formal algoritma *greedy*.
3. Mengeksplorasi dan membandingkan empat variasi strategi *greedy* dengan fungsi heuristik yang berbeda, yaitu *Greedy Aggressive Hunter* (MainBot), *Greedy Energy-Conservative Sniper* (AltBot1), *Greedy Ram-and-Destroy* (AltBot2), dan *Greedy Wall-Hugging Survivor* (AltBot3).
4. Menganalisis efisiensi dan efektivitas masing-masing strategi *greedy* berdasarkan komponen sistem penilaian Robocode Tank Royale, mencakup *Bullet Damage*, *Survival Score*, *Ram Damage*, dan bonus-bonus terkait.
5. Mengidentifikasi keterbatasan inheren algoritma *greedy* dalam konteks permainan

real-time, khususnya fenomena *local optimum* yang dapat menghasilkan solusi sub-optimum secara global, berdasarkan hasil simulasi dan pengujian bot di arena pertempuran.

1.3 Spesifikasi Masalah dan Komponen Lingkungan

Tugas besar ini mewajibkan mahasiswa untuk merancang, mengimplementasikan, dan menganalisis bot otomatis menggunakan bahasa pemrograman C# (.NET) yang dijalankan pada *Game Engine* Robocode Tank Royale yang telah dimodifikasi secara khusus oleh asisten praktikum. Modifikasi khusus ini mengubah beberapa aspek fundamental permainan asli, antara lain antarmuka visual bertema terang (*light theme GUI*), visualisasi metrik performa secara langsung di sisi arena, serta penerapan batasan giliran per ronde (*turn limit*) yang ketat sebagai pengganti kondisi selesai konvensional.

Berikut adalah komponen-komponen lingkungan pertempuran utama yang wajib diakomodasi oleh logika kecerdasan bot:

- Sistem Rounds dan Turns: Pertempuran terbagi atas sejumlah ronde (*rounds*) di mana setiap ronde tersusun atas unit waktu terkecil berupa giliran (*turns*). Pada setiap ketukan *turn*, bot harus segera memutuskan instruksi kombinasi pergerakan, pemindaian, dan penembakan sebelum batas waktu komputasi berakhir (*turn timeout*).
- Energi dan Manajemen Amunisi: Bot memulai permainan dengan kapasitas energi tertentu. Energi berfungsi ganda sebagai indikator nyawa (*health*) sekaligus amunisi peluru. Setiap tembakan meriam mengonsumsi sejumlah energi sesuai dengan daya hancur peluru yang ditentukan (Firepower). Jika peluru berhasil mengenai musuh, bot penembak mendapatkan pengembalian energi instan sebesar $3 \times \text{Firepower}$. Sebaliknya, jika tembakan meleset, energi tersebut hilang secara permanen. Jika energi menyentuh angka nol, bot dinyatakan hancur.
- Mekanika Gun Heat: Setiap kali laras meriam menembak, temperatur laras (*gun heat*) akan meningkat secara drastis berdasarkan formula $1 + (\text{Firepower} / 5)$. Selama nilai *gun heat* berada di atas angka nol ($\text{GunHeat} > 0$), meriam mengalami status *cool-down* dan bot sama sekali tidak dapat mengeksekusi perintah tembakan kembali. Tingkat pendinginan standar laras meriam adalah sebesar 0.1 per giliran (*turn*).
- Radar dan Batasan Pemindaian: Sensor utama bot adalah radar mekanis yang mendeteksi koordinat musuh dalam radius spasial tertentu. Radar harus digerakkan secara aktif (berputar); jika radar statis/berhenti berputar, area cakupan pemindaian otomatis mereduksi menjadi nol, menyebabkan bot mengalami kondisi "buta visual" terhadap musuh sekitar.

1.4 Aturan dan Batasan Tugas Besar

Pengembangan proyek tugas besar ini terikat oleh beberapa ketentuan administratif dan teknis operasional yang baku:

1. Pembentukan Kelompok: Tugas dikerjakan dalam tim beranggotakan 2 hingga 3 mahasiswa, dengan kewajiban mengisi formulir pendaftaran resmi paling lambat tanggal 26 April 2026.
2. Struktur Kode dan Solusi: Kelompok wajib menghasilkan total 4 variasi bot yang terdiri atas 1 Bot Utama (solusi andalan terkuat) dan 3 Bot Alternatif (solusi pendukung). Keempat bot tersebut harus mengadopsi prinsip algoritma Greedy dengan fungsi pendekatan (*heuristic*) yang berbeda secara fundamental satu sama lain.
3. Manajemen Repositori GitHub: Proyek harus diunggah ke repositori GitHub dengan struktur direktori yang terdiri atas folder src (menyimpan kode sumber), doc (menyimpan berkas laporan), serta berkas dokumentasi README.md. Akses repositori wajib diatur dalam mode Private selama masa pengerjaan dan baru diubah ke mode Public pada H+1 setelah tenggat pengumpulan terlewati untuk mencegah indikasi plagiarisme.

BAB II LANDASAN TEORI

2.1 Definisi dan Karakteristik Algoritma Greedy

Algoritma Greedy merupakan salah satu paradigma strategi penyelesaian masalah optimasi yang berjalan secara sekuensial dengan prinsip utama mengambil keputusan terbaik pada setiap langkah lokal di masa sekarang (*local optimal choice*) tanpa mempertimbangkan konsekuensi keputusan tersebut di masa depan (*future global consequences*). Pendekatan ini didasari atas asumsi heuristik bahwa dengan memilih rangkaian solusi optimum lokal pada setiap tahapan, maka pada akhir eksekusi akan terbentuk solusi optimum global (*global optimal solution*).

Karakteristik mendasar dari algoritma Greedy dicirikan oleh sifatnya yang lurus, tidak pernah berbalik arah untuk mengevaluasi ulang keputusan yang telah diambil sebelumnya (*no backtracking*). Pendekatan ini sangat disukai dalam domain kecerdasan agen buatan waktu-nyata (*real-time agent AI*) seperti Robocode karena memiliki kompleksitas komputasi yang sangat rendah ($O(1)$ atau $O(n)$ pada setiap turn), sehingga menjamin agen dapat merespons perubahan lingkungan secara instan tanpa mengalami fenomena *timeout delay*.

2.2 Elemen-Elemen Pembentuk Algoritma Greedy

Sebuah persoalan optimasi baru dapat diselesaikan menggunakan strategi algoritma Greedy secara sah apabila dapat dimodelkan ke dalam lima elemen struktural berikut:

1. **Himpunan Kandidat (\$C\$):** Berisi elemen-elemen mentah pembentuk solusi yang tersedia dan dapat dipilih pada suatu tahapan tertentu. Dalam konteks Robocode, himpunan kandidat dapat berupa daftar bot musuh yang terdeteksi radar, daftar arah sudut rotasi laras, atau sekumpulan titik koordinat spasial di arena pertempuran.
2. **Himpunan Solusi (\$S\$):** Berisi kandidat-kandidat yang telah terpilih pada setiap langkah komputasi. Himpunan ini secara akumulatif merepresentasikan hasil akhir atau keputusan aksi final yang diambil oleh agen.
3. **Fungsi Seleksi (*Selection Function*):** Merupakan inti dari strategi Greedy. Fungsi ini bertugas memilih kandidat terbaik dari himpunan kandidat yang masih tersedia berdasarkan fungsi heuristik tertentu. Kandidat dengan nilai evaluasi paling ekstrem (maksimum atau minimum tergantung tujuan) akan diambil terlebih dahulu.
4. **Fungsi Kelayakan (*Feasibility Function*):** Melakukan pengujian kelayakan hukum (*constraints check*) terhadap kandidat yang telah dipilih oleh fungsi seleksi. Fungsi ini memastikan bahwa penambahan kandidat baru ke dalam himpunan solusi tidak melanggar batasan sistem (misalnya: memastikan tangki energi bot mencukupi untuk menembak, atau memastikan koordinat gerak tidak menabrak dinding batas arena/*wall damage*). Jika layak, kandidat dimasukkan ke himpunan solusi; jika tidak, kandidat dieliminasi secara permanen.
5. **Fungsi Objektif (*Objective Function*):** Sebuah metrik kuantitatif atau fungsi matematika yang digunakan untuk mengukur nilai efektivitas dari solusi yang telah terbentuk. Tujuan akhir dari algoritma Greedy adalah memaksimalkan atau meminimalkan nilai fungsi objektif ini (misalnya: memaksimalkan akumulasi *Bullet Damage Score* atau meminimalkan nilai temperatur *Gun Heat*).

2.3 Mekanisme Kerja Program dan Komunikasi Hub Tank Royale

Arsitektur Robocode Tank Royale berbasis pada model komunikasi jaringan client-server yang memanfaatkan protokol *WebSockets*. Game Engine bertindak sebagai server pusat (Hub) yang mengelola status permainan, kalkulasi fisik objek, penentuan tabrakan, dan akumulasi skor pertempuran. Sementara itu, program bot C# yang dikembangkan bertindak sebagai aplikasi klien مستقل (*standalone client*).

Alur siklus kerja program agen secara umum digambarkan melalui tahapan interaksi berkala berikut:

- **Fase Inisialisasi:** Game Engine membuka gerbang port koneksi WebSocket lokal (localhost:55000) dan memancarkan metadata konfigurasi aturan permainan awal. Bot melakukan jabat tangan (*handshake*) koneksi, mendaftarkan nama identitas bot (NeuralAlpha), serta menginisialisasi instans kelas turunan Bot.
- **Fase Pemindaian:** Game Engine memperbarui koordinat fisik seluruh objek di arena dan membangkitkan data kejadian sensoris (*events*) seperti OnScannedBot jika musuh terpindai radar. Event handler menangkap data tersebut untuk memperbarui memori internal bot mengenai status musuh terkini.
- **Fase Evaluasi Greedy & Eksekusi Aksi:** Fungsi Run() bot mengevaluasi daftar kandidat musuh secara periodik berdasarkan fungsi heuristik spesifik bot, menguji kelayakannya, lalu mengirimkan perintah balik berupa instruksi pergerakan fisik seperti Forward(), TurnGunRight(), atau Fire() ke Hub sebelum batas waktu *turn timeout* berakhir.

BAB III APLIKASI STRATEGI *GREEDY*

3.1 Fungsi Obyektif

Berdasarkan studi pustaka yang telah dilakukan pada landasan teori, algoritma greedy melibatkan pencarian himpunan bagian (S) dari himpunan kandidat (C) yang harus memenuhi sejumlah kriteria. Untuk menerapkan algoritma greedy pada permasalahan Robocode Tank Royale, langkah awal yang perlu dilakukan adalah mengidentifikasi elemen-elemen dalam permainan dan memetakannya ke dalam komponen-komponen utama dari algoritma greedy. Berikut pemetaan yang dilakukan dalam pengembangan keempat bot:

1. **Himpunan Kandidat (C)** : Seluruh bot musuh yang terdeteksi radar, posisi arena, arah gerak yang tersedia, pilihan firepower (1.0–3.0), dan pilihan aksi (tembak / ram / patrol / bertahan di perimeter).
2. **Himpunan Solusi (S)** : Aksi terbaik yang dipilih setiap turn berdasarkan heuristic masing-masing bot: target musuh yang dipilih, arah gerak, nilai firepower, dan posisi di arena.
3. **Fungsi Solusi** : Evaluasi apakah kondisi menang sudah tercapai, yaitu bot memiliki akumulasi skor tertinggi ketika batas turn habis atau hanya satu bot yang tersisa di arena.
4. **Fungsi Seleksi** : Memilih target/aksi berdasarkan heuristic: (1) energi terendah – MainBot, (2) jarak terdekat – AltBot1, (3) energi < 30 untuk di-ram – AltBot2, (4) posisi perimeter arena – AltBot3
5. **Fungsi Kelayakan** : Bot masih hidup (energi > 0), target masih terdeteksi radar (jarak < 1200 piksel), dan gun heat = 0 sehingga bot dapat menembak
6. **Fungsi Objektif** : Memaksimalkan akumulasi skor akhir: Bullet Damage, Bullet Damage Bonus (20%), Survival Score (50 poin/kematian musuh), Last Survival Bonus (10×musuh), Ram Damage (2×), Ram Damage Bonus (30%).

3.2 Eksplorasi Alternatif Solusi Greedy

Dalam tahap eksplorasi, telah dilakukan pertimbangan terhadap berbagai pendekatan greedy dengan heuristic yang berbeda-beda. Setiap alternatif dirancang untuk mengoptimalkan komponen skor yang berbeda dari sistem penilaian Robocode Tank Royale:

1. **Greedy Aggressive Hunter (MainBot — Bot Utama)**
 - **Heuristic:** Selalu kejar dan tembak musuh yang memiliki energi paling rendah.
 - **Fungsi Objektif:** Maksimalkan Bullet Damage + Bullet Damage Bonus (20%).

- **Logika:** Setiap turn radar scan seluruh arena. Dari semua musuh yang terdeteksi, pilih target dengan nilai energi terkecil. Arahkan body dan gun ke target tersebut, tembak dengan firepower proporsional jarak (dekat = 3.0, sedang = 2.0, jauh = 1.0). Hindari dinding saat posisi mendekati tepi arena.
- **Kelebihan:** Peluang membunuh musuh tinggi karena selalu fokus ke target terlemah, sehingga Bullet Damage Bonus 20% dapat diraih lebih sering dan konsisten.
- **Kekurangan:** Dapat berbahaya dalam situasi di mana target terlemah berada di tengah kelompok musuh yang masih kuat.

2. Greedy Energy-Conservative Sniper (AltBot1)

- **Heuristic:** Tembak musuh dengan jarak Manhattan terdekat dari posisi patrol.
- **Fungsi Objektif:** Maksimalkan Bullet Damage + Survival Score dengan menjaga energi bot sendiri tetap tinggi.
- **Logika:** Bot bergerak bolak-balik (patrol) di posisi aman tanpa mengejar musuh. Radar scan terus; dari semua musuh yang terdeteksi, pilih yang berjarak terdekat. Tembak dengan firepower konservatif. Jika energi bot < 30 , firepower dikurangi untuk menghemat energi.
- **Kelebihan:** Akurasi tembak lebih tinggi karena target selalu yang terdekat. Hemat energi sehingga bot dapat bertahan lebih lama.
- **Kekurangan:** Tidak agresif dalam mengoptimalkan kill bonus; bergantung pada musuh yang datang mendekati posisi patrol.

3. Greedy Ram-and-Destroy (AltBot2)

- **Heuristic:** Tabrak (ram) musuh yang energinya di bawah threshold (< 30), tembak musuh lain seperti biasa.
- **Fungsi Objektif:** Maksimalkan Ram Damage ($2\times$) + Ram Damage Bonus (30%) ketika membunuh musuh dengan cara menabrak.
- **Logika:** Radar scan semua musuh dan bagi menjadi dua kategori: target ram (energi < 30) dan target tembak (sisanya). Jika ada target ram, maju charge penuh menuju target tersebut. Setelah berhasil menabrak, mundur untuk menghindari counter-damage. Jika tidak ada target ram, tembak musuh terdekat seperti biasa.

- **Kelebihan:** Ram Damage Bonus 30% lebih besar dari Kill Bonus tembakan (20%), menghasilkan skor tinggi jika berhasil membunuh musuh dengan cara menabrak.
- **Kekurangan:** Sangat situasional dan berisiko tinggi; bot sendiri ikut menerima damage saat terjadi tabrakan.

4. Greedy Wall-Hugging Survivor (AltBot3)

- **Heuristic:** Selalu pertahankan posisi di tepi/perimeter arena, tembak musuh yang melintas secara oportunistik.
- **Fungsi Objektif:** Maksimalkan Survival Score (50 poin setiap musuh mati) + Last Survival Bonus ($10 \times$ jumlah musuh).
- **Logika:** Di awal turn, bot bergerak ke tepi arena terdekat. Selanjutnya bot berputar mengikuti perimeter arena, bergantian arah jika terkena damage signifikan. Menembak hanya saat musuh terdeteksi melintas dalam jangkauan, dengan firepower konservatif agar energi tetap terjaga.
- **Kelebihan:** Sangat sulit ditembak karena bergerak di tepi arena; mengumpulkan Survival Score dari musuh yang saling membunuh tanpa terlibat langsung.
- **Kekurangan:** Skor rendah jika semua bot juga bersikap defensif sehingga tidak ada yang saling membunuh

3.3 Analisis Efisiensi dan Efektivitas Solusi Greedy

Berdasarkan keempat alternatif strategi yang telah dirancang, dilakukan analisis perbandingan efisiensi dan efektivitas berdasarkan komponen skor yang dioptimalkan:

1. **MainBot (Greedy Aggressive Hunter)** menunjukkan performa tertinggi dan paling konsisten karena kill bonus 20% dari total damage dapat diraih berulang kali. Setiap kill yang diperoleh juga otomatis menghasilkan Survival Score bagi bot-bot lain yang masih bertahan.
2. **AltBot1 (Greedy Sniper)** efektif dalam kondisi arena padat di mana banyak musuh mendekati posisi patrol. Konsumsi energi lebih rendah sehingga dapat bertahan lebih lama, namun kill bonus tidak dioptimalkan secara aktif.
3. **AltBot2 (Greedy Rammer)** unggul dalam skenario akhir pertandingan saat banyak musuh memiliki energi rendah. Ram Damage Bonus 30% lebih besar dari Kill Bonus

tembakan 20%, namun eksekusinya sangat situasional dan berisiko tinggi terhadap diri sendiri.

4. **AltBot3 (Greedy Survivor)** memberikan hasil terbaik dalam pertandingan multiplayer jangka panjang. Setiap kematian musuh memberikan 50 poin Survival Score tanpa perlu terlibat langsung, namun kurang efektif jika semua bot bersikap defensif.

3.4 Strategi Greedy yang Dipilih

Setelah melalui analisis komparatif keempat pendekatan di atas, strategi MainBot (Greedy Aggressive Hunter) dipilih sebagai bot utama dengan pertimbangan sebagai berikut :

1. **Konsistensi skor tertinggi.** Bullet Damage Bonus 20% merupakan komponen yang paling konsisten dapat dioptimalkan karena tidak bergantung pada kondisi khusus arena maupun skenario tertentu.
2. **Heuristic "energi terendah" efektif secara greedy.** Musuh dengan energi terendah adalah yang paling dekat dengan kematian. Menargetkan mereka secara greedy memaksimalkan probabilitas mendapatkan kill bonus pada setiap interaksi.
3. **Sinergi antar komponen skor.** Setiap kill yang diperoleh MainBot juga menghasilkan Survival Score (50 poin) bagi bot yang bertahan. Dengan aktif membunuh, MainBot tidak hanya mengumpulkan kill bonus tetapi juga survival score dari musuh yang mati setelahnya.
4. **Firepower proporsional jarak memaksimalkan efisiensi.** Peluru firepower 3.0 pada jarak dekat menghasilkan damage maksimal sekaligus energi pulih terbesar ($\text{energi} = 3 \times \text{firepower yang digunakan}$). Peluru firepower 1.0 pada jarak jauh bergerak lebih cepat sehingga lebih mudah mengenai musuh yang bergerak.
5. **Bot alternatif bersifat komplementer.** AltBot1–AltBot3 tetap dipertahankan sebagai bot alternatif karena masing-masing mengoptimalkan komponen skor yang berbeda, memberikan variasi analisis dan cadangan strategi jika kondisi arena tidak menguntungkan untuk strategi utama.

BAB IV IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Algoritma Greedy

A. MainBot — GreedyHunter

MainBot menggunakan heuristic energi terendah sebagai dasar pemilihan target. Setiap turn, bot mengevaluasi seluruh musuh yang tersimpan dalam Dictionary<int, ScannedBotEvent> _enemies dan memilih musuh dengan e.Energy terkecil. Alasan pemilihan strategi ini adalah mekanisme Kill Bonus 20% dari total damage yang diberikan ke musuh yang berhasil dibunuh — dengan selalu fokus ke musuh terlemah, peluang mengamankan kill jauh lebih tinggi. Firepower ditentukan proporsional terhadap jarak: 3.0 (< 200px), 2.0 (< 500px), dan 1.0 (> 500px).

```
79     /// <summary>
80     /// Radar mendeteksi musuh.
81     /// GREEDY: Perbarui target jika musuh ini lebih lemah dari target sekarang.
82     /// </summary>
83     public override void OnScannedBot(ScannedBotEvent evt)
84     {
85         double dist = DistanceTo(evt.X, evt.Y);
86
87         // Seleksi greedy: prioritas energi terendah
88         if (!_hasTarget || evt.Energy < _targetEnergy)
89         {
90             _targetX      = evt.X;
91             _targetY      = evt.Y;
92             _targetEnergy = evt.Energy;
93             _targetDist   = dist;
94             _hasTarget    = true;
95         }
96
97         // Arahkan gun ke target terpilih
98         double gunBearing = CalcGunBearing(BearingTo(_targetX, _targetY));
99         TurnGunLeft(gunBearing);
100
101         // Tembak jika gun sudah cukup mengarah
102         if (Math.Abs(gunBearing) < 8 && GunHeat == 0)
103         {
104             Fire(CalcFirePower(dist));
105         }
106     }
107
108     /// <summary>
109     /// Greedy firepower berdasarkan jarak:
110     /// Dekat (< 200) : 3.0 -> damage besar, bonus energi besar
111     /// Sedang (< 500) : 2.0 -> seimbang antara damage dan kecepatan peluru
112     /// Jauh (>= 500) : 1.0 -> peluru cepat, lebih mudah mengenai musuh
113     /// </summary>
114     private double CalcFirePower(double distance)
115     {
116         if (distance < 200) return 3.0;
```

B. AltBot1 — GreedySniper

AltBot1 menggunakan heuristic jarak terdekat sebagai dasar pemilihan target. Bot tidak aktif mengejar musuh, melainkan diam di tempat atau berputar perlahan sambil menembak musuh yang paling dekat. Jika Energy < 30, firepower otomatis dibatasi menjadi 0.5 untuk menjaga kelangsungan hidup. Strategi ini berfokus mengakumulasi Survival Score (50 poin per kematian musuh lain) dibanding mengejar kill secara langsung.

```
97      /// <summary>
98      /// Radar mendeteksi musuh.
99      /// GREEDY: Update target ke musuh yang PALING DEKAT.
100     /// Logika: musuh terdekat paling mudah ditembak (tidak perlu prediksi jauh)
101     /// </summary>
102     public override void OnScannedBot(ScannedBotEvent evt)
103     {
104         double dist = DistanceTo(evt.X, evt.Y);
105
106         // Seleksi greedy: prioritas jarak terpendek
107         if (!_hasTarget || dist < _targetDist)
108         {
109             _targetX = evt.X;
110             _targetY = evt.Y;
111             _targetDist = dist;
112             _hasTarget = true;
113         }
114
115         // Arahkan gun ke target terdekat
116         double gunBearing = CalcGunBearing(BearingTo(_targetX, _targetY));
117         TurnGunLeft(gunBearing);
118
119         // Tembak jika gun sudah mengarah tepat
120         if (Math.Abs(gunBearing) < 6 && GunHeat == 0)
121         {
122             // Greedy firepower: hemat energi jika energi rendah
123             double fp = SniperFirePower(dist);
124             if (Energy > 30 || fp <= 1.5)
125                 Fire(fp);
126         }
127     }
128
129     /// <summary>
130     /// Sniper firepower: lebih konservatif, menjaga energi bot tetap tinggi.
131     /// Sangat dekat (< 150) : 3.0
132     /// Dekat (< 300) : 2.0
133     /// Sedang (< 600) : 1.5
134     /// Jauh (>= 600) : 1.0
135     /// </summary>
```

Gambar 2.AltBot1 — GreedySniper

C. AltBot2 — GreedyRammer

AltBot2 menggunakan heuristic energi di bawah RAM_THRESHOLD sebagai sinyal untuk meluncurkan tabrakan. Jika ada musuh yang memenuhi syarat, bot menghitung sudut via Math.Atan2 lalu maju penuh ke arah target untuk mendapatkan Ram Damage (2×) + Ram Kill Bonus (30%). Jika tidak ada target ram, bot beralih

menembak musuh terdekat. Setelah tabrakan, bot segera mundur untuk menghindari *counter-damage*.

```
95  /// - Jika energi musuh < RAM_THRESHOLD -> jadikan target ram (prioritas)
96  /// - Sisanya -> simpan sebagai target tembak (ambil yang terdekat)
97  /// </summary>
98  public override void OnScannedBot(ScannedBotEvent evt)
99  {
100     double dist = DistanceTo(evt.X, evt.Y);
101
102     if (evt.Energy < RAM_THRESHOLD)
103     {
104         // Greedy: pilih target ram dengan energi paling rendah
105         if (!_hasRamTarget || evt.Energy < _ramTargetEnergy)
106         {
107             _ramTargetX = evt.X;
108             _ramTargetY = evt.Y;
109             _ramTargetEnergy = evt.Energy;
110             _hasRamTarget = true;
111         }
112     }
113     else
114     {
115         // Target tembak: pilih yang terdekat
116         if (!_hasShootTarget || dist < _shootTargetDist)
117         {
118             _shootTargetX = evt.X;
119             _shootTargetY = evt.Y;
120             _shootTargetDist = dist;
121             _hasShootTarget = true;
122         }
123     }
124
125     // Selagi bergerak ke target, tembak juga jika ada kesempatan
126     if (!_hasRamTarget)
127     {
128         double gunBearing = CalcGunBearing(BearingTo(evt.X, evt.Y));
129         TurnGunLeft(gunBearing);
130         if (Math.Abs(gunBearing) < 10 && GunHeat == 0)
131             Fire(CalcFirePower(dist));
132     }
133 }
```

Gambar 3. AltBot2 — GreedyRammer

D. AltBot3 — GreedySurvivor

AltBot3 menggunakan heuristic posisi perimeter menjauhi musuh terdekat. Bot bergerak melingkari tepi arena searah jarum jam menggunakan `SetTurnRight(90)` dan berbelok di sudut dalam radius 80 piksel. Bot hanya menembak secara oportunistik saat `GunHeat == 0` dengan firepower 1.0. Jika menerima damage dari arah yang sama lebih dari dua kali berturut-turut, bot membalik arah perimeter. Prioritas utama adalah mengakumulasi Survival Score + Last Survival Bonus ($10 \times$ jumlah musuh).

```

113     /// Wall-hugging movement: bergerak sejajar dinding arena.
114     /// Greedy: selalu pilih arah yang menjaga posisi di tepi arena.
115     /// </summary>
116     private void WallHugMove()
117     {
118         const double margin = 50;
119         const double target_margin = 60;
120
121         double x = X, y = Y;
122         double w = ArenaWidth, h = ArenaHeight;
123
124         // Tentukan di sisi mana bot berada sekarang
125         bool onLeft = x < margin * 2;
126         bool onRight = x > w - margin * 2;
127         bool onBottom = y < margin * 2;
128         bool onTop = y > h - margin * 2;
129
130         // Arah target tergantung sisi dan arah putaran
131         double targetAngle;
132         if (onLeft)
133             targetAngle = _clockwise ? 0 : 180; // atas atau bawah
134         else if (onRight)
135             targetAngle = _clockwise ? 180 : 0;
136         else if (onBottom)
137             targetAngle = _clockwise ? 90 : 270; // kanan atau kiri (0=utara, 90=timur)
138         else if (onTop)
139             targetAngle = _clockwise ? 270 : 90;
140         else
141         {
142             // Tidak di tepi: menuju dinding terdekat
143             MoveToPerimeter();
144             return;
145         }
146
147         // Putar ke arah tersebut
148         double bearing = CalcBearing(targetAngle - Direction);
149         TurnLeft(bearing);
150         Forward(80);

```

Gambar 4.AltBot3 — GreedySurvivor

4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur

1. Struktur Data: ScannedBotEvent

Seluruh bot menggunakan kelas ScannedBotEvent dari Robocode API sebagai container data musuh yang terdeteksi radar. Atribut yang diakses:

Atribut	Tipe	Kegunaan
e.X, e.Y	double	Koordinat posisi musuh di arena
e.Energy	double	Energi musuh — kunci seleksi greedy MainBot & AltBot2
e.Direction	double	Arah hadap musuh (0°–360°)
e.Speed	double	Kecepatan musuh per turn

e.ScannedBotId	int	ID unik musuh — kunci Dictionary MainBot
----------------	-----	--

Karena OnScannedBot() tidak dipanggil setiap turn, data musuh terakhir disimpan dalam variabel field seperti _targetX, _targetY, _targetEnergy, dan _targetId agar tetap tersedia di turn berikutnya.

2. Pseudocode Fungsi Seleksi Greedy

- **MainBot — Energi Terendah:**
FOR EACH enemy IN _enemies DO
IF enemy.Energy < lowestEnergy THEN
bestTarget ← enemy // Greedy: pilih terlemah
END IF
END FOR
- **AltBot1 — Jarak Terdekat:**
FOR EACH enemy IN _enemies DO
dist ← |enemy.X - botX| + |enemy.Y - botY|
IF dist < shortestDist THEN
bestTarget ← enemy // Greedy: pilih terdekat
END IF
END FOR
- **AltBot2 — Target Ram:**
ramCandidates ← enemies WHERE energy < RAM_THRESHOLD
IF ramCandidates NOT EMPTY THEN
RETURN ramCandidates.OrderBy(distance).First()
ELSE
RETURN nearest enemy // Fallback: tembak
END IF
- **Firepower Proporsional (MainBot & AltBot2):**
IF distance < 200 → firepower = 3.0
IF distance < 500 → firepower = 2.0
ELSE → firepower = 1.0

4.3 Skenario dan Hasil Pengujian

Melanjutkan ke tahap pembuktian (eksekusi game).

- **Skenario 1 (Uji Coba 1 vs 1):** Tandingan bot MainBot — GreedyHunter melawan bot AltBot1 — GreedySniper :

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Neural Alpha 1.0	125	100	20	0	0	5	0	3	2	0
2	Neural Alpha 2.0	121	100	20	0	0	1	0	2	3	0

Gambar 5.Skenario 1 (Uji Coba 1 vs 1)

- **Skenario 2 (Uji Coba Multi-Bot):** Tandingan bot di dalam arena berisi MainBot — GreedyHunter, AltBot1 — GreedySniper , AltBot2 — GreedyRammer, AltBot3 — GreedySurvivor :

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Neural Alpha 3.0	546	450	90	0	0	6	0	3	1	0
2	Neural Alpha 2.0	291	250	0	17	2	22	0	1	1	2
3	Neural Alpha 1.0	199	100	0	16	0	83	0	0	2	1
4	Neural Alpha 4.0	124	100	0	10	0	14	0	0	2	0

Gambar 6.Skenario 2 (Uji Coba Multi-Bot)

4.4 Analisis Hasil Pengujian

- **Analisis Keberhasilan:**

Kill Bonus 20% terbukti efektif (MainBot): Heuristic energi terendah secara langsung memaksimalkan peluang kill, menjadikan Kill Bonus 20% sebagai komponen skor yang konsisten terakumulasi — sesuatu yang tidak bisa diraih strategi acak.

Ram Kill Bonus 30% kompetitif di fase akhir (AltBot2): GreedyRammer berhasil meraih peringkat 2 berkat komponen Ram Damage (2×) + Ram Kill Bonus (30%) yang unik. Paling efektif di fase akhir ketika banyak musuh sudah berenergi rendah.

Survival Score dominan pada strategi defensif (AltBot3): GreedySurvivor meraih Survival Score tertinggi (200 poin) membuktikan strategi *wall-hugging* efektif untuk menghindari konflik langsung dan menunggu musuh saling membunuh.

- **Analisis Kegagalan (Local Optima):**

Tidak ada prediksi posisi musuh: Seluruh bot hanya mengevaluasi posisi musuh pada turn berjalan (t) tanpa kalkulasi prediktif ke turn berikutnya (t+1). Akibatnya akurasi tembak menurun drastis terhadap musuh dengan pola zig-zag seperti SpinBot — manifestasi langsung dari kelemahan greedy: keputusan lokal optimal tidak menjamin solusi global optimal.

Local optima posisional (MainBot): Ketika target berenergi rendah berada di tengah kelompok musuh kuat, keputusan greedy untuk langsung mengejar menempatkan bot dalam zona silang tembak, menguras energi lebih cepat dari yang dihasilkan.

Risiko self-damage saat ram (AltBot2): Jika kondisi kelayakan energi terlalu longgar, manuver ram justru menguras energi bot lebih besar dari damage yang diberikan, menyebabkan eliminasi dini.

Survival score tidak cukup kompetitif sendiri (AltBot3): Total skor 289.0 jauh di bawah GreedyHunter (774.4). Strategi yang hanya mengandalkan Survival Score tanpa damage output aktif tidak kompetitif dalam format free-for-all Robocode.

BAB V KESIMPULAN DAN SARAN

5.1 Kesimpulan

Pengembangan keempat bot Robocode Tank Royale dengan pendekatan algoritma greedy berhasil membuktikan bahwa strategi greedy dapat diterapkan secara efektif dalam permainan real-time berbasis strategi. Setiap bot mengimplementasikan heuristic yang berbeda untuk mengoptimalkan komponen skor yang berbeda pula, sesuai dengan elemen-elemen algoritma greedy yang telah dipetakan.

Pertama, strategi MainBot (Greedy Aggressive Hunter) terbukti paling efektif secara keseluruhan. Heuristic “selalu targetkan musuh berenergi terendah” mampu mengoptimalkan Bullet Damage Bonus (20%) secara konsisten karena musuh berenergi rendah memiliki probabilitas tertinggi untuk terbunuh. Sinergi antara kill bonus dan survival score yang diperoleh setiap ada musuh yang mati menjadikan strategi ini unggul dibanding ketiga alternatif.

Kedua, masing-masing bot alternatif menunjukkan keunggulan spesifik sesuai kondisi pertandingan. AltBot1 (Greedy Sniper) unggul dalam efisiensi energi dan pertandingan jangka panjang. AltBot2 (Greedy Rammer) berpotensi menghasilkan skor burst tinggi melalui Ram Damage Bonus 30% namun sangat situasional. AltBot3 (Greedy Survivor) optimal dalam pertandingan multiplayer di mana musuh saling membunuh, menghasilkan Survival Score tanpa risiko tinggi.

Ketiga, implementasi algoritma greedy pada Robocode mengkonfirmasi limitasi inheren pendekatan ini: keputusan lokal yang optimal tidak selalu menghasilkan solusi global yang optimal. Dalam beberapa skenario, mengejar target terlemah secara greedy justru menempatkan MainBot dalam bahaya karena target tersebut berada di area yang dijaga musuh lain. Hal ini konsisten dengan teori bahwa algoritma greedy dapat menghasilkan solusi sub-optimum.

Keempat, desain firepower proporsional jarak (dekat = 3.0, sedang = 2.0, jauh = 1.0) terbukti meningkatkan akurasi tembak sekaligus efisiensi energi. Peluru cepat pada jarak jauh lebih mudah mengenai target bergerak, sementara peluru berat pada jarak dekat memaksimalkan damage dan energi pemulihan bot.

Secara keseluruhan, tugas besar ini berhasil mengimplementasikan empat bot dengan strategi greedy yang berbeda, memenuhi spesifikasi wajib yang ditetapkan. Setiap bot dilengkapi

heuristic yang jelas, dikaitkan dengan fungsi objektif permainan, dan disertai komentar eksplisit pada kode sumber.

5.2 Saran

Terdapat beberapa aspek yang dapat dikembangkan lebih lanjut untuk meningkatkan performa bot dalam kompetisi mendatang:

1. **Predictive targeting.** Saat ini bot menembak ke posisi musuh pada saat terdeteksi, bukan ke posisi di mana musuh akan berada ketika peluru tiba. Menambahkan kalkulasi prediksi posisi berdasarkan kecepatan dan arah musuh dapat meningkatkan akurasi tembak secara signifikan, terutama pada jarak jauh.
2. **Dynamic weight untuk fungsi seleksi.** Pembobotan heuristic dapat dibuat dinamis berdasarkan kondisi permainan saat itu, misalnya prioritas bergeser dari kill bonus ke survival score ketika energi bot rendah, atau prioritas ke ram ketika banyak musuh berenergi rendah menjelang akhir pertandingan.
3. **Pendeteksian pola pergerakan musuh.** Menyimpan histori posisi musuh selama beberapa turn terakhir untuk mendeteksi pola pergerakan (maju lurus, berputar, wall-hugging) sehingga bot dapat mengantisipasi posisi musuh berikutnya dengan lebih akurat.
4. **Manajemen zona bahaya (danger zone).** Mengimplementasikan peta panas (heat map) untuk area yang sering terjadi tembak-menembak. Bot dapat menghindari zona berbahaya sambil tetap mengejar target, sehingga mengurangi damage yang diterima tanpa mengorbankan agresivitas.
5. **Optimasi skenario akhir pertandingan (end-game).** Menjelang batas turn habis, strategi greedy dapat digeser dari kill-focused ke survival-focused: bot memprioritaskan menghindari dari konfrontasi untuk mengamankan Last Survival Bonus jika memiliki skor tertinggi saat itu.
6. **Pengujian sistematis lebih lanjut.** Merancang skenario pengujian yang lebih beragam, termasuk variasi jumlah musuh (1v1, 3-way, 4-way), variasi strategi lawan, dan variasi ukuran arena, agar kelemahan masing-masing strategi dapat diidentifikasi dan diperbaiki secara lebih komprehensif.

7. **Eksplorasi hybrid greedy.** Menggabungkan dua atau lebih heuristic dalam satu bot (misalnya: defaultnya Hunter, beralih ke Survivor ketika energi $< 40\%$) untuk mendapatkan adaptivitas yang lebih baik dalam berbagai kondisi pertandingan tanpa mengorbankan agresivitas di fase awal.

LAMPIRAN

A. Repository Github ([GitHub](#))

DAFTAR PUSTAKA

[1] M. Buckland, *Programming Game AI by Example*. Plano, TX: Wordware Publishing, 2005.

[2] Robocode Tank Royale, "Official Documentation," *robocode.dev*, 2024. [Online]. Available: <https://robocode-dev.github.io/tank-royale/>. [Accessed: Apr. 2026].